UD3 · Prueba práctica · El navegador como parte de la app · Versión A

Módulo Desarrollo Web en Entorno Cliente (DEW)

CFGS Desarrollo de Aplicaciones Web, 2.º curso · Curso 2026/27

Nombre y apellidos	
Fecha	
Resultado de aprendizaje	RA3. Escribe código, identificando y aplicando las funcionalidades aportadas por los objetos predefinidos del lenguaje.
Criterios de evaluación	a, c, d, g, h (y b en la lectura de la URL).
Duración	45 minutos (5 de lectura + 40 de trabajo). Además, dispondrán de 20 min adicionales
Puntuación	10 puntos: Ej. 1 → 3 · Ej. 2 → 3 · Ej. 3 → 4.

Instrucciones

1. Descarga del aula virtual la carpeta prueba-ud3-a y cópiala en tu repositorio dwec-ejercicios, en ut03/prueba-ud3-a. Contiene ejercicioN.html y ejercicioN.js para cada ejercicio y estilos.css.
2. Escribe tu nombre en la cabecera de cada .js.
3. Abre cada página con **Live Server** (http://127.0.0.1:5500/...), nunca con doble clic, y con la consola abierta (F12).
4. **Material permitido:** los apuntes de la unidad (UD3_NavegadorApp), MDN y tus actividades de ut03. **No permitido:** asistentes de IA, mensajería ni compartir ficheros.
5. Normas de estilo: 'use strict', solo const y let, nombres en camelCase, funciones documentadas con **JSDoc**, y todo lo que se escribe en la página con **textContent** (nada de innerHTML ni document.write).
6. **Entrega:** git add, git commit -m "Prueba UD3 versión A" y git push, y sube la carpeta comprimida a la tarea del aula virtual. Un programa que no se ejecuta por un error de sintaxis se corrige, pero no puede alcanzar la puntuación máxima.

Organiza el tiempo: unos 10 minutos para el ejercicio 1, 12 para el 2 y 18 para el 3. Si te atascas, pasa al siguiente.

Ejercicio 1 · Predice y completa (3 puntos · CE a, c)

Fichero ejercicio1.js. Tiene dos partes.

Parte A · Predice y justifica (1 punto). Debajo de cada línea P1 a P4, escribe en el comentario qué muestra la consola y una justificación de una línea con el concepto de la unidad que lo explica (inmutabilidad de cadenas, slice con negativos, redondeos de Math, toFixed devuelve texto, desbordamiento de Date, meses desde 0, bucle de eventos…). La mitad de la puntuación de cada línea está en la justificación. Puedes comprobarlas pegándolas en la consola de las DevTools: mientras la parte B tenga huecos, el fichero no se ejecuta.

Parte B · Completa los huecos (2 puntos). El programa calcula el precio de un billete de ferry con IGIC y cuánto falta para el viaje. Sustituye cada hueco ① a ⑧ por el código que falta, sin cambiar nada más:

Hueco	Qué tiene que hacer

①	El mes de la fecha, como número y empezando en 0 (usa slice, como en anio y dia).
②	La fecha del viaje en hora local, construida con anio, mes y dia (no con el texto ISO).
③	Condición de error: precio no positivo (incluido NaN) o fecha inválida. Pista: si fechaViaje.getMonth() o getDate() no coinciden con mes y dia, la fecha ha «desbordado» (2026-02-30) o es NaN.
④	Un único formateador de Intl para euros en IDIOMA.
⑤	Formateador de fechas con dateStyle: 'full' en IDIOMA.
⑥	Días enteros entre hoy y fechaViaje (restar fechas da milisegundos; usa MS_POR_DIA y Math.round).
⑦	El texto relativo con Intl.RelativeTimeFormat y numeric: 'auto' («mañana», «dentro de 36 días»).
⑧	Escribir en #precio, #fecha y #faltan con textContent y cambiar document.title como indica el comentario.

Comprobación (el jueves 12/11/2026): 45.5 y 2026-12-18 → Precio: 45,50 € (con IGIC: 48,69 €) · Salida: viernes, 18 de diciembre de 2026 · Viajas dentro de 36 días · pestaña Ferry · 48,69 €. 1250 y 2026-11-13 → 1250,00 € (sin punto de miles: cuatro cifras) y Viajas mañana. abc, -3, 2026-02-30 o Cancelar → Datos no válidos….

Ejercicio 2 · Escribe una función (3 puntos · CE a, d, h)

En ejercicio2.js:

a) Escribe enteroAleatorio(min, max), que devuelve un entero aleatorio entre min y max, ambos incluidos.

b) Escribe crearLogin(nombreCompleto, numero), que genera un nombre de usuario:

- Validación (cláusula de guarda): nombreCompleto debe ser texto con al menos dos palabras (ignorando espacios de sobra al principio, al final y entre palabras) y numero un entero entre 1 y 99. Si algo no es válido, devuelve null.
- Login: la inicial de la primera palabra + la segunda palabra completa, todo en minúsculas y sin tildes (normalize('NFD') y el patrón de los apuntes, apartado 2.2), seguido del número con dos cifras (padStart).
- Ejemplo: crearLogin(' Ada LOVELACE byron ', 7) → 'alovelace07'.

c) Programa: pide el nombre con prompt, llama a crearLogin con un número de enteroAleatorio(1, 99) y escribe en #salida, con un ternario y textContent, Tu usuario: … o Datos no válidos (también si se cancela).

d) Documenta las dos funciones con JSDoc y completa la tabla de pruebas del final del fichero llamando a crearLogin con esos valores fijos (por ejemplo, con console.log o console.table).

Ejercicio 3 · Amplía la página (4 puntos · CE a, b, c, d, g, h)

ejercicio3.html muestra un reloj que ya funciona (no lo modifiques). Añade en ejercicio3.js una cuenta atrás:

a) formatearMMSS(segundos) (JSDoc): devuelve el tiempo como MM:SS, con Math.trunc, % y padStart. 65 → '01:05' · 600 → '10:00' · 0 → '00:00'.

b) iniciarCuenta() (JSDoc), que se ejecuta al pulsar «Iniciar» (la conexión con el botón ya está hecha):

- Lee los segundos del parámetro segundos de la URL con URLSearchParams; si no viene o no es un entero positivo, usa 30.

- Cancela la cuenta anterior si ya había una en marcha (pulsar «Iniciar» dos veces no puede crear dos intervalos), usando la variable idCuenta.
- Pinta el valor inicial inmediatamente en #cuenta y después, cada segundo, resta uno y actualiza #cuenta y document.title (⏳ 00:12).
- Al llegar a cero: muestra ¡Tiempo!, cancela el intervalo y restaura el título original (TITULO_INICIAL).

c) Persistencia: cada vez que una cuenta llega a cero, incrementa en localStorage la clave cuentasCompletadas y actualiza #completadas. Al cargar la página, muestra en #completadas el valor guardado (0 si no existe). Compruébalo en Application → Local Storage y recargando la página.

Comprobación: abre ejercicio3.html?segundos=5, pulsa «Iniciar»: 00:05, 00:04… ¡Tiempo! y el contador sube en uno; recarga y el contador se mantiene. Sin parámetro empieza en 00:30. Pulsa «Iniciar» dos veces seguidas: la cuenta no debe ir el doble de rápido.

d) Para sobresaliente (1 punto). La página ya tiene un <h2 id="saludo">, un y un botón «Borrar historial» conectado a borrarHistorial().

- d1 · Saludo desde la URL. Si la URL trae el parámetro nombre (y no está vacío tras trim), escribe en #saludo ¡Ánimo, <nombre>! con textContent. Pruébalo con ?nombre=Ada: las etiquetas deben verse como texto, no en negrita. Explica en un comentario por qué.
- d2 · Historial en JSON. Cada vez que una cuenta llega a cero, guarda en localStorage, en la clave ultimaCuenta, un objeto con la duración (segundos) y la fecha y hora (new Date().toISOString()) usando JSON.stringify. Escribe mostrarUltima() (JSDoc), que lo lee con JSON.parse y escribe en #ultima, por ejemplo, 00:05, el 12 de noviembre de 2026 a las 10:15 (duración con formatearMMSS y fecha con un formateador de Intl con dateStyle: 'long' y timeStyle: 'short'). Llámala al cargar la página. Si no hay datos o el JSON está corrupto (edítalo a mano en Application y borra una llave), debe mostrar — sin romper la página (try…catch).
- d3 · Borrar historial. Escribe borrarHistorial() (JSDoc): elimina solo las claves cuentasCompletadas y ultimaCuenta (removeItem, no clear) y deja la página como recién cargada (contador a 0 y —).

Comprobación de d): ejercicio3.html?segundos=3&nombre=Ada → saludo con las etiquetas visibles; al terminar, #ultima muestra la duración y la fecha; recarga y se mantiene; JSON roto → —; «Borrar historial» → 0 y —, y en Application desaparecen las dos claves.

Antes de entregar: ¿has puesto tu nombre en los tres ficheros? ¿Se ejecutan sin errores en la consola? ¿Has subido el zip al aula virtual?